REPRESENTATION NUMBERS OF QUADRATIC FORMS AND TUNNELL'S CONGRUENT-NUMBER TEST IN TIME $n^{o(1)}$, ASSUMING THE NEW #SAT ALGORITHMS

VAIBHAV TULSYAN

Abstract. For a fixed positive-definite integer matrix A, the representation number $r_A(n) = \#\{x \in \mathbb{Z}^k : x^\top Ax = n\}$ is classical. Four ternary representation numbers decide Tunnell's congruent-number test, and through Waldspurger's theorem such numbers give central values of twisted L-functions. However, for a general ternary form the best deterministic bound we know for a single $r_A(n)$ is $n^{1/2+o(1)}$, and we know of no bound of the form $n^{o(1)}$, even conditionally. We present a deterministic algorithm that outputs $r_A(n)$ in time $2^{O(L/\sqrt{\log L})} = n^{o(1)}$, where L is the bit length of n, for every fixed positive-definite A of any dimension. Our approach shifts x into a box of naturals by an adjugate Cauchy–Schwarz bound, splits signs so that $Q(u - B) = n$ becomes an identity between two nonnegative straight-line programs, pads the input so that all values fit, and counts the solutions with Anthropic's counting engine through our earlier network library. We prove three theorems in Lean 4 (`repNum_TM_rate`, `diagRep_TM_rate`, `tunnell_TM_rate`; axioms `propext`, `Classical.choice`, `Quot.sound`): a finite-alphabet Mathlib Turing machine outputs the general count, the diagonal count, and Tunnell's test bit within $\lfloor C \cdot 2^{CL/\sqrt{\log_2(L+2)}} \rfloor$ steps. On paper, Tunnell's Theorem 3 then gives $L(E_n, 1)$ exactly for squarefree n, and a theorem of Böcherer and Schulze-Pillot gives central values of imaginary quadratic twists of a fixed curve of squarefree conductor, in time $n^{o(1)}$ and $|D|^{o(1)}$ respectively; if OpenAI's unrefereed full-BSD result holds, the counts also give the order of the Tate–Shafarevich group of E_n in analytic rank 0. The constants are astronomical, the machines use exponential space, and everything rests on Anthropic's Lean development of its claimed refutation of the Exponential Time Hypothesis, which is under review.

1. Introduction

How many ways can an integer be written by a quadratic form? For a positive-definite integer matrix $A \in \mathbb{Z}^{k\times k}$ we write

$$Q_A(x) = x^\top Ax, \qquad r_A(n) = \#\{x \in \mathbb{Z}^k : \ Q_A(x) = n\}.$$

These numbers are old. Gauss related $r_A(n)$ for $x^2 + y^2 + z^2$ to class numbers, and Jacobi expressed the four-square count through divisor sums. They are also current. Tunnell [26] showed that four ternary representation numbers decide whether a squarefree n is the area of a rational right triangle, up to the Birch and Swinnerton-Dyer conjecture (BSD). His proof goes through Waldspurger's theorem, which relates the coefficients of weight-3/2 theta series to central values of twisted L-functions. Gross [7] and Böcherer and Schulze-Pillot [3] made this explicit for many curves.

Computing one $r_A(n)$ is not easy. Enumerating the ternary box takes $O(n^{3/2})$ steps; Tunnell wrote that his criterion reduces the congruent-number problem "to an algebraic computation involving $O(D^{3/2})$ steps" [26, p. 325]. Fixing one variable and counting representations of $n - cz^2$ by binary forms gives about $n^{1/2+o(1)}$ steps for one n, and $O(N^{3/2})$ for all $n \le N$ [9, §1.3]. Hart, Tornaría and Watkins computed Tunnell's counts for all $n \le 10^{12}$ by fast multiplication of theta series [9], which is quasi-linear in the range but not faster for one n. However, for a general ternary form we know of no deterministic algorithm for a single $r_A(n)$ in time $n^{o(1)}$, even conditionally.

In October 2026 OpenAI [18] and Anthropic [1] (below [O] and [A]) each claimed a deterministic algorithm that counts the satisfying assignments of a k-CNF in subexponential time, refuting the Exponential Time Hypothesis; both claims come with Lean 4 developments and are under review. Rudich and Rousseau [22] used the engines to factor in $N^{o(1)}$, Shunia [24] proved the rate $2^{O(L/\sqrt{\log L})}$ on paper, and our factoring

Date: 5 November 2026.

2020 *Mathematics Subject Classification.* 11Y16, 11E25, 11G05, 11F67, 68Q25, 03B35.

Key words and phrases. Representation numbers, ternary quadratic forms, congruent numbers, Tunnell's theorem, central L-values, quadratic twists, exact counting, Exponential Time Hypothesis, Lean 4.

note [30] proved it in Lean on [A]'s engine. Our class-number note [28] built a library, ArithNet, that evaluates counts of solutions of systems with few products, and our parity note [31] extended it. We found no derivation of representation numbers, Tunnell's counts or these L-values from either engine (Section 11).

1.1. **The problem and the running example.** We consider the *representation-number problem*: for a fixed k and a fixed positive-definite $A \in \mathbb{Z}^{k\times k}$, given $n \in \mathbb{N}$ in binary, output $r_A(n)$ in binary. We also consider Tunnell's counts. With the names of our Lean development, let

$$A_n = r_{x^2+2y^2+8z^2}(n), \quad B_n = r_{x^2+2y^2+32z^2}(n), \quad C_n = r_{8x^2+2y^2+16z^2}(n), \quad D_n = r_{8x^2+2y^2+64z^2}(n),$$

and let *Tunnell's test* be the bit $[A_n = 2B_n]$ for odd n and $[C_n = 2D_n]$ for even n.

Example 1.1 (Running example)**.** Take $n = 3$. The solutions of $x^2 + 2y^2 + 8z^2 = 3$ are $(\pm 1, \pm 1, 0)$, so $A_3 = 4$; likewise $B_3 = 4$. The test asks whether $4 = 8$; it is false. By Tunnell's Theorem 3 (Section 6) this means $L(E_3, 1) = (A_3 - 2B_3)^2\beta/(16\sqrt{3}) = \beta/\sqrt{3} \approx 1.5138$ for the curve $E_3 : y^2 = x^3 - 9x$, with $\beta \approx 2.62206$ a fixed period. So E_3 has rank 0, and 3 is not congruent.

1.2. **The naive route and the principle.** At first one might enumerate the box of candidate vectors. However, the box has $\Theta(n^{k/2})$ points. One might instead reduce the equation to #SAT and apply the engines' headline theorems. However, Tseitin's transformation of a multiplication circuit has $\Omega(L \log L)$ clauses for every known circuit, and [A]'s clause bound then gives $2^{O(L\sqrt{\log L})}$, which is superpolynomial in n; our factoring note makes this point in detail [30, §1]. ArithNet avoids it, but it counts *natural* solutions of *nonnegative* linear combinations of products. A quadratic form has integer variables and may have negative entries.

Instead, we rely on a different principle: *a positive-definite form is coercive, so after a shift every solution is a vector of naturals in a box of side $O(n)$, and after splitting the entries of A into positive and negative parts, $Q_A(u - B) = n$ is an equality between two sums of products with natural coefficients.* Such an equality is a straight-line program with $O(k^2)$ products, which ArithNet counts with $O(L)$ shared bits. The values are about n^2, larger than the input word allows, so we pad the input with zeros; this changes the rate by a constant factor only.

1.3. **What we prove.** In this paper we build the straight-line description of $Q_A(u - B) = n$, prove that its count is $r_A(n)$, pad it, and prove in Lean that one Turing machine per form outputs the answer.

Theorem 1.2 (Section 5; machine-checked)**.** *Let $L = |\texttt{encodeNat}\ n|$ be the length of the binary word of n. There are Turing machines in Mathlib's `TM2` model, with finite alphabets on all stacks, and constants $C \in \mathbb{N}$, such that for every $n \in \mathbb{N}$ the machine halts on `encodeNat` n within $\lfloor C \cdot 2^{CL/\sqrt{\log_2(L+2)}} \rfloor$ steps with the following output:*

(i) `repNum_TM_rate`*: for every k and every positive-definite $A \in \mathbb{Z}^{k\times k}$ (one machine and one C per A), the word* `encodeNat` $r_A(n)$*;*

(ii) `diagRep_TM_rate`*: for every k and all $a_1, \dots, a_k \geq 1$, the word* `encodeNat` $\#\{x \in \mathbb{Z}^k : \sum_i a_i x_i^2 = n\}$*;*

(iii) `tunnell_TM_rate`*: one machine, the one-bit word of Tunnell's test.*

Since $L = \lfloor \log_2 n \rfloor + 1$ for $n \geq 1$, the bound is $n^{O(1/\sqrt{\log\log n})} = n^{o(1)}$, against the trivial $O(n^{k/2})$ and, for ternary forms, $n^{1/2+o(1)}$. To our knowledge, no deterministic algorithm with a bound of the form $n^{o(1)}$ was known for a general ternary form, even conditionally. Section 9 says which special forms were already within reach.

1.4. **Consequences on paper.** The theorem computes counts, and the meaning of the counts is classical. On paper, and not formalised, we derive four consequences in Sections 6 to 9.

(1) For squarefree n, Tunnell's Theorem 3 [26] gives $L(E_n, 1)$ exactly from the counts, so $L(E_n, 1)\sqrt{n}/\beta \in \mathbb{Q}$ is computed in time $n^{o(1)}$. If the test is false, n is not congruent, unconditionally. If it is true and $n \equiv 1, 2, 3 \pmod 8$, the analytic rank is at least 2, and only then is BSD needed to conclude that n is congruent. For $n \equiv 5, 6, 7 \pmod 8$ the counts vanish identically.

(2) For a fixed elliptic curve E of squarefree conductor N with $L(E, 1) \neq 0$, the theorem of Böcherer and Schulze-Pillot [3] writes $L(E, 1)L(E \otimes \chi_{-D}, 1)\sqrt{D}$ as a constant times m_D^2, where m_D is a fixed rational

combination of $O(1)$ ternary representation numbers, for the D that satisfy fixed residue conditions. So these central values are computed exactly in time $|D|^{o(1)}$.

(3) If OpenAI's catalogue entry 002 [16] is correct (it is unrefereed), Tunnell's conjecture $|Ш(E_n)| = (a(n)/\sigma_0(n))^2$ holds for every odd squarefree n with $L(E_n, 1) \neq 0$, and its analogue for even n, so $|Ш(E_n)|$ is also computed in time $n^{o(1)}$.

(4) Some special cases were already within reach: binary forms, even rank, and forms alone in their genus reduce to factoring and known methods (for ternary genera, with class numbers, which our class-number note computes in the same time). The new content is odd rank, where the cusp part of the theta series carries central L-values.

1.5. **Contributions.** Our contributions are:

- We introduce a sign-split straight-line description of $Q_A(u - B) = n$ with $16k^2 + 3$ steps, one product and one chain of coefficient mass 3 per step, and prove that its count is $r_A(n)$ once B exceeds the adjugate box bound (Section 3).
- We present a generic padding transformer for counting packages, which lets a description use values of $2L + O(1)$ bits on an input of L bits at a constant-factor cost in the exponent (Section 4).
- We prove the three theorems of Theorem 1.2 in Lean and list what a reader must trust (Sections 5 and 10).
- We derive on paper the consequences for Tunnell's test, for central values of imaginary quadratic twists and, conditionally, for Tate–Shafarevich groups, and we say which special forms were already within reach (Sections 6 to 9).

The method is simple: a box bound, a shift, a sign split and a padding step. The network library and the machines are those of our class-number note [28]; what we add is the description, its counting proof, the padding, and the bridge to the number theory.

1.6. **Status of each claim.** We separate four kinds of claim. *Machine-checked in Lean*: the three theorems of Theorem 1.2, with the axioms `propext`, `Classical.choice` and `Quot.sound` only, and every lemma whose Lean name we give. *Proved on paper only*: everything in Sections 6 to 9, which connects the counts to congruent numbers, L-values and Tate–Shafarevich groups; the reduction of forms with odd cross coefficients to Theorem 1.2 by doubling (Remark 3.4); and the comparisons with earlier bounds. *Assumed*: for Theorem 1.2, nothing beyond the three axioms, since [A]'s results enter as Lean theorems of its development, which the kernel checks with ours. For the consequences on paper, the cited theorems of the literature; BSD where we say so; and, for Section 8 only, OpenAI's unrefereed catalogue entry 002 [16]. *Checked numerically*, by short exact computations that are not part of any proof (PARI/GP 2.17.2 [21] and Python): the running examples; Tunnell's L-value formula for all 183 squarefree $n \leq 300$; the vanishing of the counts for $n \equiv 5, 6, 7 \pmod 8$ in that range; Tunnell's formula for $|Ш|$ against the analytic order of Ш for the 73 squarefree $n \leq 300$ with $L(E_n, 1) \neq 0$; Gross's formula for the curve $11a$ for all fundamental $-D$ with $D < 140$ and $(\frac{-D}{11}) = -1$; and Gauss's formula for $r_3(n)$ for squarefree $5 \leq n < 800$.
How we read the literature. We read in the original: Tunnell [26], Böcherer and Schulze-Pillot [3], Pal [20], Mao, Rodriguez-Villegas and Tornaría [14], Pacetti and Tornaría [19], Hart, Tornaría and Watkins [9], Conrad [5] and OpenAI's catalogue entries [16, 17]. We could not obtain Gross's paper [7] and use the restatement in [14, (3.1)]. We mark with † the results that we cite from the standard literature without having re-read the source for this note.

Outline. Section 2 recalls [A]'s counting theorem and our library. Section 3 turns a form into a description and Section 4 pads it. Section 5 quotes the statements. Sections 6 to 9 give the consequences on paper. Section 10 describes the Lean development, and Sections 11 to 13 cover related work, limitations and conclusions.

2. The engine and the network library

In this section we recall the one theorem of [A] that we compose with and the interface of ArithNet, both from our earlier notes [30, §2] and [28, §§2–4]. We use [A]'s Lean development at commit `16a5ce4` (`anthropics/formal-math`, folder `eth`, Lean v4.33.1, Mathlib `0df444a`).

A *network on* $[g]$ *with parameters* P, B, τ has, for every node a, settings of at most P bits and a weight W_a, and for every pair of nodes an interaction through $c_{a,b}$ shared bits with $\sum_b c_{a,b} \leq P$. Its value is

$Z = \sum_z \prod_a W_a(z_a) \prod_{\{a,b\}} \Psi_{a,b}$ and its total is $U = \sum_{a \neq b} c_{a,b}$. We use only label systems with 0/1 tables, whose value is the number of accepted labellings.

Imported Theorem 2.1 ([A], Lemma 8(i) and Theorem A(b); see [30, Imported Thm. 2.2]). *Let $K = K_\ell \geq 2$ be a level of* [A]*'s schedule, $g = 3K^8$, and $e_\ell \leq (3x_0 + 6)/\sqrt{\log_2 K}$ for an absolute x_0. A network on $[g]$ with parameters P, B, τ and total U can be evaluated within $\big(K^g + 2^{e_\ell K^2 L'}(K(L' + B'))^{C_0 K^2} 2^{2gP/K} \tau\big)(g(P+B))^{C_1}$ operations, where $L' = \lceil (U + gP/K)/K^2 \rceil$, $B' = \lceil 2gP/K \rceil + g^2 B$ and C_0, C_1 are absolute.*

So a network family of size s with $U = O(s)$ and $gP = O(s)$ is evaluated in $2^{O(s/\sqrt{\log s})}$ steps at the level $K^{40} \lesssim s$ of [30, §5].

Descriptions. ArithNet [28, §3] takes a *description*: variables $x_0, x_1, \ldots$ ranging over $[0, 2^{n_B})$, products $x_u \cdot w$ of a variable by a variable or a constant, chains $\sum c_t \operatorname{val}_t = \sum c'_t \operatorname{val}'_t$ with natural coefficients of mass at most 32 per side, comparisons, and bit patterns. Its count at size s is the number of assignments in $[0, 2^{n_B})^{\mathrm{nV}}$ that satisfy everything, where the box width $n_B > s$ is fixed by the size. The parsimony theorem (`arithNet_value`) says that the network value is this count, and the parameter bound (`arithSys_U_le`) gives $U \leq 2^{11} c_A(D)(s+2)$ with the shape constant $c_A(D) = 64(\mathrm{nV} + \mathrm{nP} + \mathrm{nQ} + 1)(\mathrm{nC} + 1)$. A polynomial-time map from input words to well-formed descriptions with bounded c_A gives a counting package (`ArithSpec.pkg_exists`), and the one- and two-query machines (`GR.queryM_TM`) turn a package into a Turing machine with the rate of Theorem 1.2. Everything in this paragraph is machine-checked in [28] and reused unchanged.

3. From a quadratic form to a nonnegative straight-line program

In this section we bound the solutions, shift them into the naturals, split the signs, and write the result as a straight-line program. All of it is in the pure Mathlib modules `RepQF`, `SlpDesc` and `RepDesc`. Throughout, $A \in \mathbb{Z}^{k \times k}$ is positive definite in the sense of Mathlib's `Matrix.PosDef`, so in particular symmetric, and $Q = Q_A$.

3.1. The box.

The first step is a bound on each coordinate that is linear in $Q(x)$ and has an integer constant.

Lemma 3.1 (`posDef_sq_le`, `posDef_coercive`). *For every $x \in \mathbb{Z}^k$ and every j, $x_j^2 \leq |\operatorname{adj}(A)_{jj}| \cdot Q(x)$. Hence $x_j^2 \leq c(A)\, Q(x)$ with $c(A) = \sum_j |\operatorname{adj}(A)_{jj}|$.*

Proof. Let $y = \operatorname{adj}(A) e_j$, so $Ay = \det(A) e_j$. Then $x^\top A y = \det(A) x_j$ and $y^\top A y = \det(A) \operatorname{adj}(A)_{jj}$. Cauchy–Schwarz for the positive form (`cs`) gives $\det(A)^2 x_j^2 \leq Q(x) \det(A) \operatorname{adj}(A)_{jj} \leq Q(x)\, |\det A|\, |\operatorname{adj}(A)_{jj}|$. Since $\det A$ is a nonzero integer, $|\det A| \leq \det(A)^2$, and the claim follows. □

For an integer, $|x_j| \leq x_j^2$, so every solution of $Q(x) = n$ lies in the box $[-cn, cn]^k$ with $c = c(A)$ (`repNum_eq_card_box`). This is crude, since $|x_j| \leq \sqrt{cn}$ also holds, but the padding of Section 4 absorbs the extra bits.

3.2. The shift.

Let $B = cn + 1$. The map $x \mapsto u = x + B$ is a bijection between the solutions of $Q(x) = n$ and the vectors $u \in [0, B_x)^k$ with $Q(u - B) = n$, for every box size $B_x \geq 2B$, and all such u lie in $[1, 2B)^k$ (`card_shift_two`, `shift_bounds`). The machine for diagonal forms with $a_i \geq 1$, and hence Tunnell's machine, uses $c = 1$ (`coercive_diagonal`).

3.3. The sign split.

Write $A = A^+ - A^-$ with $A^\pm_{ij} = \max(\pm A_{ij}, 0)$. For each entry the identity

$$A^+_{ij} u_i u_j + A^-_{ij} B u_i + A^-_{ij} B u_j + A^+_{ij} B^2 \\ = A^-_{ij} u_i u_j + A^+_{ij} B u_i + A^+_{ij} B u_j + A^-_{ij} B^2 + A_{ij}(u_i - B)(u_j - B)$$

holds over $\mathbb{Z}$. Summing over i, j, the left sides give a sum $S_L(u)$ of $4k^2$ monomials $c\, x_p x_q$ with natural c, where $x_i = u_i$ for $i < k$ and $x_k = B$, and the right sides give $S_R(u) + Q(u - B)$ with S_R of the same shape. So

$$Q(u - B) = n \iff S_L(u) = n + S_R(u) \qquad (\texttt{monos_shift}). \tag{3.1}$$

Both sides of (3.1) are natural numbers with nonnegative coefficients; no subtraction remains.

Example 3.2 (Example 1.1, continued)**.** For $A = \mathrm{diag}(1,2,8)$ only diagonal entries are nonzero and $A^- = 0$, so (3.1) reads $\sum_i a_i(u_i^2 + B^2) = n + \sum_i 2a_iBu_i$. Tunnell's machine has $c = 1$, so $B = n + 1 = 4$, and the four solutions become $u \in \{3,5\} \times \{3,5\} \times \{4\}$. At $u = (5,5,4)$ both sides equal 379: the left side is $(25 + 16) + 2(25 + 16) + 8(16 + 16)$ and the right side is $3 + 8(5 + 10 + 32)$. The general machine of Theorem 1.2(i) uses $c(A) = 16 + 8 + 2 = 26$ and $B = 79$ instead.

3.4. **The straight-line program.** A *step* computes $z := (x_a \text{ or } 0) + x_u \cdot w + \text{const}$, where w is a variable or a constant. It is encoded as one product $x_u \cdot w$ and one chain $1 \cdot x_a + 1 \cdot (x_u \cdot w) + 1 \cdot \text{const} = 1 \cdot z$, of mass 3. A program of steps defines the description `slpDesc`: the k free variables, one auxiliary variable per step, and final comparisons. When every step reads only free variables and earlier auxiliaries, each free tuple has exactly one satisfying run (`slpRun_unique`), so the count is the number of free tuples in the box whose run satisfies the comparisons, provided that the run's values fit in the box (`slpDesc_count`).

Our program (`repProg`) first sets $x_k := B$. Then each side of (3.1) is an accumulator: for the monomials $c_m x_{p_m} x_{q_m}$ it computes $y_m := c_m \cdot x_{p_m}$ and $\mathrm{acc}_{m+1} := \mathrm{acc}_m + x_{q_m} \cdot y_m$, starting from 0 on the left and from n on the right. The final comparison asks that the two accumulators be equal. There are $1 + 2(1 + 8k^2) = 16k^2 + 3$ steps, so $\mathrm{nV} = k + 16k^2 + 3$, $\mathrm{nP} = \mathrm{nC} = 16k^2 + 3$ and $\mathrm{nQ} = 1$; the shape constant is $c_A = 64\,(k + 2(16k^2 + 3) + 2)(16k^2 + 4)$ (`repDesc_cA_eq`), a constant for fixed k.

Proposition 3.3 (`repDesc_count`)**.** *Let $k \geq 1$, $x_j^2 \leq cQ(x)$ for all x, $|A_{ij}| \leq c_m$, $cn < B$, and $K = 16(k^2+1)(c_m+1)$. If $n + KB^2 < B_x$ and $n + KB^2 < 2^{b\ell}$ (the limb capacity), then the count of the description in $[0, B_x)$ is the number of solutions of $Q(x) = n$ in $[-cn, cn]^k$, which is $r_A(n)$.*

The proof combines the shift bijection, (3.1), and the bound $n + KB^2$ on every value of the run (`sumProg_bound`). Well-formedness is unconditional (`repDesc_WF`).

Remark 3.4 (Forms with odd cross coefficients). Theorem 1.2 is about $x^\top Ax$ with A symmetric and integral, so its cross coefficients $2A_{ij}$ are even. An integer-valued form $Q(x) = \sum_i a_{ii}x_i^2 + \sum_{i<j} a_{ij}x_ix_j$ with some a_{ij} odd has $2Q = x^\top Gx$ for its integral Gram matrix G, and $r_Q(n) = r_G(2n)$. A machine for r_Q then doubles its input and runs the machine for G. This is on paper only; the forms in Sections 6 and 7 that we write out have even cross coefficients.

4. Padding the input so that the values fit

In this section we explain why the description cannot be counted on the input word itself, and how a generic padding step fixes this. The modules are `CntPad` and `RepPkg`.

The package of a description family sets the size from the input word w, and the box width n_B grows linearly with $|w|$. For the input `encodeNat` n we have $n < 2^L$, but the run of Proposition 3.3 reaches $n + K(cn + 1)^2$, which needs about $2L$ bits. So the box condition of Proposition 3.3 fails on the input word. We pad instead. For a constant c_3, let

$$\mathrm{padW}_{c_3}(w) = w\,0^{|w|+c_3},$$

so that $|\mathrm{padW}_{c_3}(w)| = 2|w| + c_3$ and the little-endian value is unchanged (`gofWord_padW`). The padded package (`CntPackage.padded`) answers the query of the original package on the padded word, and keeps every bound with the constant $c_0 \cdot (c_3+2)2^{\lfloor \log_2(c_3+2)\rfloor+2}$; the depth term uses $d(2L+c_3) \leq 2^{\lfloor \log_2(c_3+2)\rfloor+2}\, d(L)$ (`dg_pad`). The rate theorem for the padded package (`GR.queryM_TM_padded`) has its hypotheses stated on the original one.

Lemma 4.1 (`size_bound`, `repFam_Q_pad`)**.** *Let $c_3 = \lfloor \log_2(1+K(c+1)^2)\rfloor+1$. If $n < 2^L$, then $n+K(cn+1)^2 < 2^{2L+c_3}$. Hence, on the padded word, the count of the description is $\#\{x : Q(x) = n\}$ for every n.*

The network now has size $s = 2L + c_3 = O(L)$, and c_A is a constant, so $U, gP = O(L)$ and Imported Theorem 2.1 gives $2^{O(L/\sqrt{\log L})}$. The machine for (i) asks one query (`specRep`). The machine for (iii) asks two, one for each form of the parity of n, and outputs $[Q_1 = 2Q_2]$ (`specTun`); it answers $n = 0$ directly, where `tunnellTest` 0 is false. For $k = 0$ the answer $[n = 0]$ needs no count (`repNum_TM_zero`). The final theorems are `repNum_TM_of` and `tunnell_TM_of`, instantiated in `FinalRep`.

5. What a reader must trust

A reader who wants to trust Theorem 1.2 must check the following definitions and nothing else of ours. They are in the pure module `RepQF`; we render Unicode symbols as mathematics. Here $\cdot_v$ is Mathlib's dot product (`dotProduct`) and $*_v$ its matrix–vector product (`Matrix.mulVec`).

```
def qf {k : ℕ} (A : Matrix (Fin k) (Fin k) ℤ) (x : Fin k → ℤ) : ℤ := x ⬝ᵥ A *ᵥ x
noncomputable def repNum {k : ℕ} (A : Matrix (Fin k) (Fin k) ℤ) (n : ℕ) : ℕ :=
  Set.ncard {x : Fin k → ℤ | x ⬝ᵥ A *ᵥ x = n}
def tForm (a b c : ℕ) : Matrix (Fin 3) (Fin 3) ℤ := Matrix.diagonal ![(a : ℤ), b, c]
noncomputable def tunnellTest (n : ℕ) : Bool :=
  if n % 2 = 1 then decide (repNum (tForm 1 2 8) n = 2 * repNum (tForm 1 2 32) n)
  else decide (repNum (tForm 8 2 16) n = 2 * repNum (tForm 8 2 64) n)
```

`Set.ncard` is 0 on infinite sets, but by Lemma 3.1 the set is finite for positive-definite A, and Lean proves that it equals the finite box count (`repNum_eq_card_box`). `repNum` does not use `qf`; we list `qf` because the coercivity bound behind Lemma 3.1 is stated with it. The final statements, verbatim from `FinalRep` up to line breaks (proofs omitted), are:

```
theorem repNum_TM_rate (k : ℕ) (A : Matrix (Fin k) (Fin k) ℤ) (hA : A.PosDef) :
    ∃ (M : Turing.TM2ComputableAux Bool Bool) (C : ℕ), (∀ i, Finite (M.tm.Γ i)) ∧
      ∀ n : ℕ, GraphProblems.Outputs M (Computability.encodeNat n)
        (Computability.encodeNat (repNum A n))
        ⌊(C : ℝ) * 2 ^ ((C : ℝ) * (Computability.encodeNat n).length /
          Real.sqrt (Real.logb 2 ((Computability.encodeNat n).length + 2)))⌋₊
theorem diagRep_TM_rate (k : ℕ) (a : Fin k → ℕ) (ha : ∀ i, 0 < a i) :
    ∃ (M : Turing.TM2ComputableAux Bool Bool) (C : ℕ), (∀ i, Finite (M.tm.Γ i)) ∧
      ∀ n : ℕ, GraphProblems.Outputs M (Computability.encodeNat n)
        (Computability.encodeNat (Set.ncard {x : Fin k → ℤ | ∑ i, (a i : ℤ) * x i ^ 2 = n}))
        ⌊(C : ℝ) * 2 ^ ((C : ℝ) * (Computability.encodeNat n).length /
          Real.sqrt (Real.logb 2 ((Computability.encodeNat n).length + 2)))⌋₊
theorem tunnell_TM_rate :
    ∃ (M : Turing.TM2ComputableAux Bool Bool) (C : ℕ), (∀ i, Finite (M.tm.Γ i)) ∧
      ∀ n : ℕ, GraphProblems.Outputs M (Computability.encodeNat n) [tunnellTest n]
        ⌊(C : ℝ) * 2 ^ ((C : ℝ) * (Computability.encodeNat n).length /
          Real.sqrt (Real.logb 2 ((Computability.encodeNat n).length + 2)))⌋₊
```

The binary encoding is Mathlib's (`Mathlib.Computability.Encoding`), little-endian with the empty word for 0:

```
def encodePosNum : PosNum → List Bool
  | PosNum.one => [true]
  | PosNum.bit0 n => false :: encodePosNum n
  | PosNum.bit1 n => true :: encodePosNum n
def encodeNum : Num → List Bool
  | Num.zero => []
  | Num.pos n => encodePosNum n
def encodeNat (n : ℕ) : List Bool := encodeNum n
```

`Outputs` is the one-line definition of [A]'s statement file `GraphProblemsStatement`, which we also trust:

```
def Outputs (M : TM2ComputableAux Bool Bool) (x y : List Bool) (T : ℕ) : Prop :=
  Nonempty (TM2OutputsInTime M.tm (x.map M.inputAlphabet.invFun)
    (some (y.map M.outputAlphabet.invFun)) T)
```

Behind these are Mathlib's `Matrix.PosDef`, `Matrix.diagonal`, `dotProduct`, `Matrix.mulVec`, `Set.ncard`, `TM2OutputsInTime`, `Real.logb` and `Real.sqrt`, from Mathlib `0df444a`. Note that `tunnellTest` is defined for every n, while Tunnell's theorem is about squarefree n; congruent numbers and L-functions do not occur in the statements. The constant C depends on A and is not tracked.

6. Tunnell's test and exact central values

In this section we state what Tunnell's counts mean, and what follows from Theorem 1.2. Everything here is on paper. Let $E_n : y^2 = x^3 - n^2x$. A squarefree n is *congruent*, the area of a right triangle with rational sides, exactly when $E_n(\mathbb{Q})$ is infinite [26, p. 324].

Conventions. Tunnell writes $a(n) = \#\{2x^2+y^2+32z^2 = n\} - \frac{1}{2}\#\{2x^2+y^2+8z^2 = n\}$ for odd n and defines b through the forms of the even case [26, pp. 325, 328]. With our names,

$$a(n) = B_n - \tfrac{1}{2}A_n \quad (n \text{ odd}), \qquad b(n/2) = D_n - \tfrac{1}{2}C_n \quad (n \text{ even}).$$

Many statements of the criterion swap our A_n and B_n, and our C_n and D_n, and write $2A_n = B_n$; the test is the same.

Cited Theorem 6.1 (Tunnell [26, Theorem 3])**.** *For squarefree n,*

$$L(E_n,1) = \frac{(A_n - 2B_n)^2\,\beta}{16\sqrt{n}} \quad (n \text{ odd}), \qquad L(E_n,1) = \frac{(C_n - 2D_n)^2\,\beta}{8\sqrt{n}} \quad (n \text{ even}),$$

where $\beta = \int_1^\infty dx/\sqrt{x^3-x} = 2.62205\ldots$ is the real period of E_1 in Tunnell's normalisation.

Tunnell states it as $L(E_d,1) = a(d)^2\beta d^{-1/2}/4$ and $L(E_{2d},1) = b(d)^2\beta(2d)^{-1/2}/2$ for odd squarefree d; the display is the same formula in our names. We checked it against PARI/GP for all 183 squarefree $n \le 300$ (maximal absolute difference $4.3 \cdot 10^{-12}$).

Corollary 6.2 (on paper)**.** *For squarefree n, the rational number $L(E_n,1)\sqrt{n}/\beta$ is computed in time $n^{o(1)}$, by two runs of the machines of Theorem 1.2(ii).*

If n is not known to be squarefree, its squarefree part is found by factoring n, in time $2^{O(L/\sqrt{\log L})}$ by [24, 30]; n is congruent exactly when its squarefree part is.
The congruent-number test. The root number of E_n is $+1$ for $n \equiv 1,2,3 \pmod 8$ and -1 for $n \equiv 5,6,7 \pmod 8$ (squarefree n). This is consistent with Tunnell's remark that $a(n) = 0$ unless $n \equiv 1,3 \pmod 8$ and $b(m) = 0$ unless $m \equiv 1,5 \pmod 8$ [26, p. 329].

Corollary 6.3 (on paper)**.** *Let n be squarefree.*

(a) If Tunnell's test is false, then n is not congruent. This is unconditional: $L(E_n,1) \neq 0$ by Cited Theorem 6.1, so $E_n(\mathbb{Q})$ is finite by Coates and Wiles [4]†, whom Tunnell quotes for this step [26, p. 329].

(b) If $n \equiv 5,6,7 \pmod 8$, the counts vanish identically: $A_n = B_n = 0$ for odd n and $C_n = D_n = 0$ for even n. So the test is true and carries no information.

(c) If $n \equiv 1,2,3 \pmod 8$ and the test is true, then $L(E_n,1) = 0$ with root number $+1$, so the analytic rank of E_n is even and at least 2. BSD then predicts rank at least 2, and n congruent. This is the only case in which the test's verdict "congruent" needs BSD.

Proof of (b). For odd n in $x^2+2y^2+8z^2 = n$ or $x^2+2y^2+32z^2 = n$, x is odd, so $x^2 \equiv 1$ and the left side is 1 or 3 modulo 8. For even n in $8x^2+2y^2+16z^2 = n$ or $8x^2+2y^2+64z^2 = n$, $n/2 = 4x^2+y^2+8z^2$ or $4x^2+y^2+32z^2$ is odd because n is squarefree, so y is odd and $n/2 \equiv 1 \pmod 4$, that is $n \equiv 2 \pmod 8$. In all other classes the counts are 0. □

For $n \equiv 5,6,7 \pmod 8$, analytic rank 1 already implies that n is congruent, unconditionally, by Gross–Zagier [8]† and Kolyvagin [11, 12]†. But our machines do not compute $L'(E_n,1)$, so they decide nothing in these classes.

Example 6.4 (Example 1.1, continued; two true tests)**.** For $n = 41 \equiv 1 \pmod 8$ we get $A_{41} = 32$ and $B_{41} = 16$, so the test is true and $L(E_{41},1) = 0$. Here Corollary 6.3(c) needs BSD; 41 is in fact congruent, with sides $\frac{40}{3}, \frac{123}{20}, \frac{881}{60}$. For $n = 34 \equiv 2 \pmod 8$ we get $C_{34} = 8$ and $D_{34} = 4$, and 34 is congruent with sides $\frac{15}{2}, \frac{136}{15}, \frac{353}{30}$. We found both triangles by a short search for points on E_n and checked them exactly.

7. Central values of imaginary quadratic twists

In this section we fix an elliptic curve and vary the twist. Everything here is on paper. Gross [7] proved, for prime conductor, that the central values of the imaginary quadratic twists are squares of coefficients of a fixed combination of ternary theta series; Böcherer and Schulze-Pillot [3] extended this to squarefree level.

Setting [3, §1.4, §3]. Let $N>1$ be squarefree and let $f\in S_2(\Gamma_0(N))$ be a normalised newform, an eigenform of all Hecke operators, with sign $+$ in the functional equation of $L(f,s)$. Let w_q be its Atkin–Lehner eigenvalues, $N_1=\prod_{w_q=-1}q$ and $N_2=N/N_1$. Let $\mathcal{D}$ be the definite quaternion algebra ramified exactly at the primes dividing N_1 and at ∞, let R be an Eichler order of level N in $\mathcal{D}$ with h classes represented by $y_1,\dots,y_h$, let R_i be the corresponding orders and $e_i=|R_i^\times|$, and let $L_i=\{x\in\mathbb{Z}+2R_i:\operatorname{tr}(x)=0\}$ with the norm form, a positive-definite ternary lattice. Let φ be the function on the classes attached to f, and

$$W(\varphi)=\sum_{i=1}^{h}\frac{\varphi(y_i)}{e_i}\,\theta(L_i,z),\qquad a(W;d)=\sum_{i=1}^{h}\frac{\varphi(y_i)}{e_i}\,r_{L_i}(d).$$

Cited Theorem 7.1 (Böcherer and Schulze-Pillot [3, §3, (3.5)])**.** *In this setting, let $-d_0<0$ be a fundamental discriminant and $N_0=N/(N,d_0)$. Then*

$$c_2\,d_0^{1/2}\prod_{q\mid N_0}\big(1+(\tfrac{-d_0}{q})w_q\big)\,L(f,1)\,L\big(f,(\tfrac{-d_0}{\cdot}),1\big)=c_1\,D_f^{(N)}(1)\,a(W;d_0N_0^2)^2,$$

where c_1,c_2 are explicit constants depending only on N and $D_f^{(N)}(1)$ is essentially the Petersson norm of f. If N is odd, $a(W;d_0N_0^2)^2=a(W;d_0)^2$.

For N prime this is Gross's Proposition 13.5, which we know through the restatement of [14, (3.1)].

Corollary 7.2 (on paper)**.** *Let $E/\mathbb{Q}$ have squarefree conductor N and $L(E,1)\neq0$, and let f be its newform. Let $-D<0$ be a fundamental discriminant with $(D,N)=1$ and $(\frac{-D}{q})=w_q$ for every $q\mid N$. Then*

$$L(E\otimes\chi_{-D},1)=\frac{\kappa_E\,m_D^2}{\sqrt{D}},\qquad \kappa_E=\frac{c_1D_f^{(N)}(1)}{2^{\omega(N)}c_2\,L(E,1)},\qquad m_D=\sum_{i=1}^{h}\frac{\varphi(y_i)}{e_i}\,r_{L_i}(DN^2),$$

with κ_E independent of D. So m_D, and hence $L(E\otimes\chi_{-D},1)$ up to the fixed constant κ_E, is computed exactly in time $|D|^{o(1)}$ by $h=O(1)$ runs of the machine of Theorem 1.2(i).

Proof. $L(E,1)\neq0$ forces the sign $+$. Since $(D,N)=1$, $N_0=N$, and each factor of the product is 2. Divide Cited Theorem 7.1 by $2^{\omega(N)}c_2\sqrt{D}\,L(E,1)$. Each r_{L_i} is the representation number of a fixed positive-definite ternary form, at the argument DN^2 of bit length $O(L)$. The bilinear form $-\operatorname{tr}(xy)$ on the trace-zero part of $\mathbb{Z}+2R_i$ takes even values, so its Gram matrix is integral and Theorem 1.2 applies directly. Checking that $-D$ is fundamental needs the factorisation of D, which takes time $2^{O(L/\sqrt{\log L})}$ [24] and does not change the bound. □

The rational value. With Pal's period relation [20, Main Result 1.1, Cor. 2.6], for squarefree $d<0$ coprime to the discriminant of E, $\Omega(E^d)=\tilde{u}\,c_\infty(E^d)\,\Omega^-(E)/\sqrt{|d|}$, where c_∞ counts real components, Ω^- is the imaginary period and $\tilde{u}$ is a power of 2, equal to 1 when $d\equiv1\pmod4$. (For $4\mid D$ we apply this to $d=-D/4$, since the twist by $-D$ is the twist by $-D/4$.) So $L(E^{-D},1)/\Omega(E^{-D})$ is m_D^2 times a constant that depends only on E and $\tilde{u}$. Since this ratio is rational for every twist†, the constant is rational, and there are finitely many of them; each can be fixed once from one D with $m_D\neq0$. Pal notes that the formula without $\tilde{u}$ is misquoted in parts of the literature [20].

Scope. Corollary 7.2 covers only the D that satisfy the residue conditions. Pacetti and Tornaría describe the method of [3] as covering "only a fraction of imaginary quadratic twists (determined by quadratic residue conditions)", and give the composite-level version used in computations [19]. For prime N, the excluded D coprime to N have root number -1 by the standard formula for the root number of a twist†, so their central values vanish. Real twists and base curves with $L(E,1)=0$ need the generalised theta series of [14], with weights on residue classes; these are counts of representations by fixed inhomogeneous quadratic polynomials, which Theorem 1.2 does not cover as stated.

Example 7.3 (The curve $11a$)**.** For $E=11a$ ($y^2+y=x^3-x^2-10x-20$), $N=11$, $w_{11}=-1$ and $h=2$. Mao, Rodriguez-Villegas and Tornaría write the theta series of the eigenvector as the difference of the theta series of

$$Q_1=4x_1^2+11x_2^2+12x_3^2+4x_1x_3,\qquad Q_2=16x_1^2+15x_2^2+15x_3^2+14x_2x_3+28x_1x_3+16x_1x_2$$

[14, (4.1), (4.2)]. Both have even cross coefficients and determinant 484, so Theorem 1.2(i) applies to them directly. With $m_D=\frac12(r_{Q_1}(D)-r_{Q_2}(D))$, Table 1 shows that $L(E,1)\,L(E\otimes\chi_{-D},1)\sqrt{D}/m_D^2$ is the

same number, 0.7406174494…, for every fundamental $-D$ with $D < 140$, $(\frac{-D}{11}) = -1$ and $m_D \neq 0$; for $D = 47, 103, 119$ both m_D and the L-value vanish.

D	r_{Q_1}	r_{Q_2}	m_D	$L(E \otimes \chi_{-D}, 1)$	D	r_{Q_1}	r_{Q_2}	m_D	$L(E \otimes \chi_{-D}, 1)$
3	0	2	−1	1.68450	59	8	6	1	0.37984
4	2	0	1	1.45882	67	0	6	−3	3.20801
15	4	6	−1	0.75333	71	16	18	−1	0.34626
20	4	6	−1	0.65240	91	8	0	4	4.89362
23	8	6	1	0.60837	103	12	12	0	0
31	8	6	1	0.52402	104	16	12	2	1.14439
47	12	12	0	0	111	20	18	1	0.27693
56	8	12	−2	1.55954	115	4	6	−1	0.27207

TABLE 1. The curve 11a: representation numbers of the two forms of Example 7.3 and the central values of the twists, from PARI/GP. The table stops at $D = 115$; the check ran to $D < 140$. $L(E, 1) = 0.2538418609$.

8. Tate–Shafarevich groups, conditionally

In this section we assume one unrefereed result and say what follows. Everything here is on paper.

OpenAI's catalogue entry 002, "The full BSD formula from low Selmer corank" [16], consists of preprints dated 6–7 October 2026. Its manuscript abstract states: "We prove the full Birch–Swinnerton-Dyer leading-term formula for every elliptic curve over $\mathbb{Q}$ whose full q-power Selmer group has corank zero or one at some prime q. The analytic and Mordell–Weil ranks equal that corank, and the Tate–Shafarevich group is finite." We have not checked these preprints, and they are not refereed.

If $L(E, 1) \neq 0$, then $E(\mathbb{Q})$ and Ш(E) are finite by Kolyvagin [11, 12]†, so the q-power Selmer group has corank 0 for every q, and entry 002 would give the full BSD formula for E. The catalogue itself advertises only the consequence for a density-one set of twists, with entry 006 [17]; the pointwise statement for analytic rank 0 is our reading of the abstract above.

Proposition 8.1 (on paper; conditional on entry 002)**.** *Let n be squarefree and Tunnell's test false. If entry 002 is correct, then*

$$|Ш(E_n)| = \Big(\frac{A_n - 2B_n}{2\,\tau(n)}\Big)^2 \quad (n \text{ odd}), \qquad |Ш(E_n)| = \Big(\frac{C_n - 2D_n}{2\,\tau(n/2)}\Big)^2 \quad (n \text{ even}),$$

where τ counts divisors. So $|Ш(E_n)|$ is computed in time $n^{o(1)}$.

Proof. In Tunnell's notation the right sides are $(a(n)/\sigma_0(n))^2$ and $(b(n/2)/\sigma_0(n/2))^2$, which is Tunnell's conjecture [26, p. 330]. Tunnell's remark there computes the local data of E_n ($c_p = 4$ for odd $p \mid n$, $c_2 \in \{2, 4\}$, $|E_n(\mathbb{Q})_{\mathrm{tors}}| = 4$) and identifies his conjecture with the BSD formula, given Cited Theorem 6.1. By the paragraph above, entry 002 gives that formula. For the cost, $\tau(m) = \sum_{j \le m} \tau(j) - \sum_{j \le m-1} \tau(j)$ is two runs of the machine `divisorSum_TM_rate` of our class-number note [28], which is machine-checked; factoring [24, 30] gives it as well. □

We checked the formula against the analytic order of Ш that PARI/GP computes from $L(E_n, 1)$, the real period, the torsion and the Tamagawa numbers, for the 73 squarefree $n \le 300$ with $L(E_n, 1) \neq 0$; all agree, and $|Ш| > 1$ occurs 27 times (for example $|Ш(E_{17})| = 4$, $|Ш(E_{113})| = 16$). The odd part of Tunnell's conjecture was known before, from Rubin's work on curves with complex multiplication [23]†; we did not re-check its exact scope.

Remark 8.2 (General twists). For the twists of Corollary 7.2, entry 002 would give $|Ш(E^{-D})| = (L/\Omega)\,|E^{-D}(\mathbb{Q})_{\mathrm{tors}}|^2 / \prod_p c_p$. The Tamagawa numbers at $p \mid D$ need the primes dividing D, so factoring enters, in time $2^{O(L/\sqrt{\log L})}$ by [24, 30]. We have not worked out the local data in general, so we state no theorem.

Entry 002 does not change the status of the congruent-number problem itself. In Corollary 6.3(c) the analytic rank is at least 2, and entry 002 assumes Selmer corank at most 1; it says nothing there.

9. What was already within reach

In this section we say plainly which forms Theorem 1.2 does not make newly computable. Everything here is on paper, and "within reach" means: in time $n^{o(1)}$ given the engines and known methods.

Binary forms. For a fixed binary form, $r_A(n)$ is computed in time polynomial in L from the factorisation of n, since the class group of the fixed discriminant is fixed and finite. Factoring takes $2^{O(L/\sqrt{\log L})}$ with the engines [22, 24, 30].

Even rank. For $k = 4$, Jacobi's formula gives $r_{x^2+y^2+z^2+w^2}(n) = 8\sigma(n)$ for odd n, so Theorem 1.2(ii) computes $\sigma(n)$. Given the factorisation, $\sigma(n)$ is easy; conversely, $\sigma(n)$ yields the factorisation in randomised polynomial time by Bach, Miller and Shallit [2]†. So the engine is at least as strong as factoring, which is consistent with [22]. More generally, for even k the theta series is an Eisenstein series, whose coefficients are twisted divisor sums and need only the factorisation, plus a cusp form of integral weight. For weight 2 the cusp coefficients at primes come from point counts on the associated abelian varieties; for higher weight, Edixhoven, Couveignes and coauthors compute Ramanujan's $\tau(p)$ in probabilistic polynomial time [6]. So even-rank forms reduce essentially to factoring and known methods; for weight above 2 this is modulo the reach of such methods, which we did not check for the cusp forms that occur here, and modulo their being probabilistic. Our theorem adds a deterministic route there, not new reach.

Forms alone in their genus. For $x^2 + y^2 + z^2$, Gauss's formula gives, for squarefree $n \geq 5$, $r_3(n) = 12h(-4n)$ for $n \equiv 1, 2, 5, 6 \pmod 8$, $24h(-n)$ for $n \equiv 3 \pmod 8$, and 0 for $n \equiv 7 \pmod 8$ (we checked it for squarefree $5 \leq n < 800$ against counts of primitive reduced forms). So Theorem 1.2(ii) with $a = (1, 1, 1)$ computes these class numbers in time $n^{o(1)}$. Our class-number note [28] already computes $h(-N)$ in the same time, through reduced forms, so the two results are consistent and neither adds reach to the other. For a ternary form alone in its genus, Siegel's theorem expresses $r_A(n)$ through the genus, whose coefficients are, up to local factors that depend on the factorisation of n, class numbers of imaginary quadratic orders, as Gauss's formula illustrates; we state this only as a pointer, without the exact local factors.

Odd rank: the new part. For a ternary form with more than one class in its genus, the theta series also has a cusp part of weight 3/2. By Waldspurger's theorem†, of which Cited Theorems 6.1 and 7.1 are explicit cases, the squares of its coefficients at squarefree n are, up to fixed factors, central values of twisted L-functions. We know of no method that computes these in time $n^{o(1)}$ for a single n, even with the engines, other than the one presented here. This is where Theorem 1.2 adds reach; Sections 6 and 7 are examples.

10. The Lean development

In this section we list the modules and state the trusted base. The development is in the public repository [32]: directories `lean/EthFollowup/ModP` (pure Mathlib; the files compile with Lean v4.33.1 and v4.34.1) and `lean-bridge-anthropic/bridge/EthFollowup/ModP` (on top of [A]'s development). Table 2 lists the new modules.

module	content	lines
`RepQF`	`qf`, `repNum`, Cauchy–Schwarz, the adjugate box, the shift, `tForm`, `tunnellTest`	318
`SlpDesc`	straight-line-program descriptions, the canonical run, its count	337
`RepDesc`	monomials, the sign split (3.1), the accumulators, Proposition 3.3	600
`CntPad`	padding of counting packages; the padded rate theorem	244
`RepPoly`	polynomial time of the descriptions	110
`RepPkg`	the family package, Lemma 4.1, query specifications, the machines	320
`FinalRep`	the three final theorems	76
total		2005

Table 2. The new modules, with line counts (`wc -l`). The first three are pure Mathlib. The development reuses, unchanged, the ArithNet, package and machine modules of [28] (`ArithDesc`, `ArithParams`, `ArithNet`, `ArithPackage`, `CntPackage`, `GRate`, ...), the factoring modules of [30] and the bridge modules of [27].

Trusted base and axioms. `#print axioms` reports `propext`, `Classical.choice` and `Quot.sound` for the three theorems of Theorem 1.2; we ran it on 5 November 2026 against a build of the pinned development whose sources are identical to the files of Table 2. No new module contains `sorry`, a new axiom or `native_decide`. A fresh `leanchecker` replay of the dependency closure of `EthFollowup.ModP.FinalRep`, the module holding the three final theorems, passed on 5 November 2026. What must be trusted is the Lean kernel, the Mathlib definitions named in Section 5, our definitions quoted there, and [A]'s definition of `Outputs`. [A]'s development, Mathlib's proofs and our other modules are checked by the kernel; they are part of the checked closure, not of the trusted base.

One design point. The padding is generic: `CntPackage.padded` works for any counting package whose rounds admit the original query words, and the rate theorem for it is proved once. Padding the input word, instead of enlarging the box inside the description family, leaves ArithNet and its parameter proofs untouched.

11. Related work

We organise related work by idea: computing representation numbers and Tunnell's counts, central values of twists, the congruent-number problem, and the counting engines with their formal proofs.

Representation numbers and Tunnell's counts. Tunnell noted the $O(n^{3/2})$ cost of his criterion [26, p. 325]. Counting representations of $n - cz^2$ by binary forms brings one n to about $n^{1/2+o(1)}$, and Hart, Tornaría and Watkins put the cost of this route up to N at $O(N^{3/2})$ [9, §1.3]. They computed Tunnell's counts for all $n \le 10^{12}$ by out-of-core FFT multiplication of theta series, split by residue classes following Elkies, in about 30 hours for the classes $1, 3 \pmod 8$ on a 16-core machine [9]. That is quasi-linear in the range, so $X^{o(1)}$ per coefficient on average, but with memory about X and no gain for one n. In contrast, we compute one n in $n^{o(1)}$, for every fixed positive-definite form, at the price of galactic constants.

Central values of twists. The approximate functional equation computes $L(E \otimes \chi_D, 1)$ numerically in time "very roughly proportional to $|D|$" [14]. Mao, Rodriguez-Villegas and Tornaría compute the coefficients of the theta combination for all $|D| \le x$ in about $x^{3/2}$ steps, by running over the lattice points of ellipsoids, and extend the method to real twists and to $L(E, 1) = 0$ with generalised theta series [14]; Pacetti and Tornaría treat composite levels [19]. Closest to our work are these papers, whose formulas we use. In contrast, we evaluate one coefficient in $|D|^{o(1)}$ instead of enumerating the ellipsoid.

The congruent-number problem. Tunnell's criterion [26] is unconditional in one direction by Coates and Wiles [4]†; Conrad gives an exposition [5]. For $n \equiv 5, 6, 7 \pmod 8$ much is known unconditionally; for example, Smith proves that at least 55.9% of squarefree $n \equiv 5, 6, 7 \pmod 8$ are congruent [25]. OpenAI's entries 002 and 006 [16, 17] give, if correct, the full BSD formula in Selmer corank at most 1 and Goldfeld's density statements for twists; neither decides the analytic-rank-≥ 2 case of Corollary 6.3(c). In contrast, our contribution to this problem is the speed of the test, not its logical status.

Counting engines and formal proofs. [O] [18] and [A] [1] claim deterministic subexponential #SAT algorithms, with Lean developments, under review. Rudich and Rousseau [22] factored in $N^{o(1)}$ with them, Shunia [24] proved the rate $2^{O(L/\sqrt{\log L})}$ on paper, and [10] and our factoring note [30] proved it in Lean. Our earlier notes applied [A]'s engine to Hamiltonian cycles [27], Eulerian circuits [29], class numbers and lattice-point counts [28], and the parity of $\pi(x)$ [31]. The closest is the class-number note, whose library we reuse; in contrast, the equations here have $\Theta(k^2)$ products, integer unknowns and values of $2L$ bits, which the shift, the sign split and the padding handle. We use Lean 4 [13] and Mathlib [15]; Mathlib at the commit we use proves Lagrange's four-square theorem, and we found in it no representation counts and no congruent numbers.

Novelty search. On 5 November 2026 we grepped the titles and abstracts of all 716 works on Hexagon through its public query interface for representation numbers, ternary and quadratic forms, theta series, congruent numbers, Tunnell, Waldspurger, BSD, central values and elliptic curves; queried arXiv's interface for the Exponential Time Hypothesis or #SAT together with quadratic forms, congruent numbers and Tunnell; and grepped OpenAI's catalogue for the same terms. This complements our novelty audit of the same day [32] (file `research/novelty-audit-2026-11-05.md`). We found no work that computes representation numbers, Tunnell's counts or these L-values with either ETH engine. A negative search result is weak evidence.

12. Limitations

- *The results stand or fall with* [A]*'s development.* Every theorem rests on Anthropic's Lean development of its claimed refutation of the Exponential Time Hypothesis, and the labs' claims are under review. Lean

checks that development as part of our proof, and our statements use only Mathlib, our quoted definitions and [A]'s `Outputs`. We do need the development, built from source at the pinned commit, to be sound, and the toolchain to be sound. The fresh kernel replay of `EthFollowup.ModP.FinalRep` passed on 5 November 2026.

- *The consequences are on paper only.* Lean proves that the machines output counts. The link to congruent numbers, L-values and Tate–Shafarevich groups (Sections 6 to 8) rests on cited theorems, some of which we did not re-read (marked †), and is not formalised. Corollary 6.3(c) needs BSD.
- *Section 8 is conditional on an unrefereed result.* OpenAI's entry 002 is a set of preprints that we have not checked.
- *The twist corollary has limited scope.* It needs $L(E,1) \neq 0$, squarefree conductor and the residue conditions of Corollary 7.2, and the constant κ_E is fixed but not computed by our machines. Real twists need weighted counts that Theorem 1.2 does not cover.
- *The algorithms are galactic.* [A]'s levels start at $K_0 = 2^{x_0}$ with $x_0 \geq 18$, the level needs $K^{40} \lesssim s$, and the constants are astronomical: for $k = 3$ the shape constant alone is $64 \cdot 299 \cdot 148 = 2{,}832{,}128$, before the factor 2^{12} of the package. The machines are of no practical use and beat $n^{1/2}$ only for astronomically large n.
- *The machines use exponential space.*
- *No human has checked the work.* The Lean code and this text were written with AI assistance. No human expert has yet checked that Section 5 lists everything a reader must trust, or the derivations on paper.

13. Conclusion

We have shown that a representation number of a fixed positive-definite form is one count of a straight-line program with nonnegative coefficients, after a shift and a sign split, and that the counting engine evaluates it in $n^{o(1)}$ steps once the input is padded. The arithmetic meaning is classical: Tunnell's counts give $L(E_n, 1)$, and ternary theta combinations give central values of twists. The result is a step towards, not a practical method for, these computations. An interesting direction is whether the same route handles the weighted counts of generalised theta series, which would cover real twists and base curves of positive rank, and whether a count can certify $L'(E_n, 1) \neq 0$, which would make the test informative for $n \equiv 5, 6, 7 \pmod 8$.

Acknowledgements

This work was done with AI assistance (Claude Code, Anthropic): the constructions, the Lean development, the numerical checks, the literature and citation checks and the text were produced in sessions with Claude Code. The Lean theorems are checked by the Lean kernel; everything else awaits human expert review. Responsibility for any errors rests with the author. This note continues the project of [27, 30, 28, 31].

References

[1] Claude (Anthropic), *A $2^{o(n)}$ time algorithm for 3-SAT*, Hexagon preprint hexagon:2610.00577v1, October 2026, https://hexagonmath.org/abs/2610.00577v1. Lean formalisation in the repository https://github.com/anthropics/formal-math, folder `eth`; we use commit `16a5ce41304fcdaea4e0be7c132e1c361dc26da5`.
[2] E. Bach, G. Miller, and J. Shallit, *Sums of divisors, perfect numbers and factoring*, SIAM J. Comput. **15** (1986), 1143–1154.
[3] S. Böcherer and R. Schulze-Pillot, *On a theorem of Waldspurger and on Eisenstein series of Klingen type*, Math. Ann. **288** (1990), 361–388.
[4] J. Coates and A. Wiles, *On the conjecture of Birch and Swinnerton-Dyer*, Invent. Math. **39** (1977), 223–251.
[5] K. Conrad, *The congruent number problem*, Harvard College Math. Review **2** (2008).
[6] B. Edixhoven and J.-M. Couveignes (eds.), *Computational aspects of modular forms and Galois representations*, Annals of Math. Studies, vol. 176, Princeton Univ. Press, 2011; arXiv:math/0605244.
[7] B. H. Gross, *Heights and the special values of L-series*, Number Theory (Montreal, 1985), CMS Conf. Proc., vol. 7, Amer. Math. Soc., Providence, RI, 1987, pp. 115–187.
[8] B. H. Gross and D. B. Zagier, *Heegner points and derivatives of L-series*, Invent. Math. **84** (1986), 225–320.
[9] W. Hart, G. Tornaría, and M. Watkins, *Congruent number theta coefficients to 10^{12}*, Algorithmic Number Theory (ANTS IX), Lecture Notes in Comput. Sci., vol. 6197, Springer, 2010, pp. 186–200.
[10] 1337isnot1337 (GitHub user) and contributors, *integer-factoring: deterministic integer factoring bounds*, public repository https://github.com/1337isnot1337/integer-factoring; Lean formalisation in `formal-appendix/` (pull request #20, merged 30 October 2026).
[11] V. A. Kolyvagin, *Finiteness of $E(\mathbb{Q})$ and Ш$(E,\mathbb{Q})$ for a subclass of Weil curves*, Izv. Akad. Nauk SSSR Ser. Mat. **52** (1988), no. 3, 522–540.
[12] V. A. Kolyvagin, *Euler systems*, The Grothendieck Festschrift, Vol. II, Birkhäuser, 1990.

[13] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, Automated Deduction – CADE 28, Lecture Notes in Comput. Sci., vol. 12699, Springer, 2021, pp. 625–635.
[14] Z. Mao, F. Rodriguez-Villegas, and G. Tornaría, *Computation of central value of quadratic twists of modular L-functions*, Ranks of Elliptic Curves and Random Matrix Theory, London Math. Soc. Lecture Note Ser., vol. 341, Cambridge Univ. Press, 2007, pp. 273–288; arXiv:math/0606762.
[15] The mathlib Community, *The Lean mathematical library*, Proc. 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), 2020, pp. 367–381.
[16] OpenAI, *Exact Birch–Swinnerton-Dyer formula from low Selmer corank*, preprint dated 7 October 2026, with companion preprints of 6–7 October 2026; entry 002, "The full BSD formula from low Selmer corank", of the catalogue `CONTENTS.md` in the repository https://github.com/openai/math. Not refereed.
[17] OpenAI, entry 006, "Goldfeld's conjecture: densities and mean analytic rank", of the catalogue `CONTENTS.md` in the repository https://github.com/openai/math, 2026. Not refereed.
[18] OpenAI, *The Exponential Time Hypothesis is false*, preprint dated 17 October 2026, revised 25 October 2026. Paper and Lean formalisation in the repository https://github.com/openai/math.
[19] A. Pacetti and G. Tornaría, *Computing central values of twisted L-series: the case of composite levels*, Experiment. Math. **17** (2008), 459–472; arXiv:math/0607008.
[20] V. Pal, *Periods of quadratic twists of elliptic curves*, with an appendix by A. Agashe, Proc. Amer. Math. Soc. **140** (2012), 1513–1525; arXiv:1012.0094.
[21] The PARI Group, *PARI/GP version 2.17.2*, Univ. Bordeaux, https://pari.math.u-bordeaux.fr/.
[22] I. Rudich and L.-M. Rousseau, *Deterministic factoring in time* $N^{o(1)}$, Hexagon preprint hexagon:2610.00600v1, October 2026, https://hexagonmath.org/abs/2610.00600v1.
[23] K. Rubin, *The "main conjectures" of Iwasawa theory for imaginary quadratic fields*, Invent. Math. **103** (1991), 25–68.
[24] J. Shunia, *An improved subexponential bound for deterministic integer factorization*, Hexagon preprint hexagon:2610.00711v1, November 2026, https://hexagonmath.org/abs/2610.00711v1.
[25] A. Smith, *The congruent numbers have positive natural density*, arXiv:1603.08479, 2016.
[26] J. B. Tunnell, *A classical Diophantine problem and modular forms of weight* 3/2, Invent. Math. **72** (1983), 323–334.
[27] V. Tulsyan, *Counting Hamiltonian cycles in sparse graphs in subexponential time, assuming the new #SAT algorithms*, Hexagon preprint hexagon:2610.00672v1, October 2026, https://hexagonmath.org/abs/2610.00672v1.
[28] V. Tulsyan, *Class numbers, square roots and quadratic non-residues in time* $2^{O(n/\sqrt{\log n})}$*, assuming the new #SAT algorithms*, Hexagon preprint hexagon:2611.00115, November 2026, https://hexagonmath.org/abs/2611.00115. Version 2 is retitled *Class numbers and lattice-point counts in time* $2^{O(n/\sqrt{\log n})}$*, assuming the new #SAT algorithms.*
[29] V. Tulsyan, *Counting Eulerian circuits in subexponential time, assuming the new #SAT algorithms*, Hexagon preprint hexagon:2611.00088v1, November 2026, https://hexagonmath.org/abs/2611.00088v1.
[30] V. Tulsyan, *An independent machine-checked proof of deterministic factoring in time* $2^{O(n/\sqrt{\log n})}$*, via the new #SAT algorithms*, preprint, November 2026; source in the directory `factoring-note/` of [32].
[31] V. Tulsyan, *The parity of the prime-counting function in time* $x^{o(1)}$*, assuming the new #SAT algorithms*, Hexagon preprint hexagon:2611.00116v1, November 2026, https://hexagonmath.org/abs/2611.00116v1.
[32] V. Tulsyan, *eth-hamiltonicity: Lean 4 formalisation and research notes*, public repository https://github.com/xennygrimmato/eth-hamiltonicity, 2026; Lean sources as of commit `3e181e7`.